

CHAPTER ONE

INTRODUCTION

Chapter One

Introduction

1.1 Introduction

In the contemporary digital era, a web presence is no longer a luxury but a fundamental necessity for businesses, creators, and individuals. However, the process of web development has traditionally been divided into two distinct worlds: professional coding, which offers infinite flexibility but requires deep technical expertise, and traditional drag-and-drop builders, which offer ease of use but often at the cost of creative freedom and code quality.

The Template-Based Website Builder (TWB) project is engineered to bridge this divide by offering a high-performance workspace that facilitates the creation of professional websites without the necessity of manual coding. By leveraging advanced web technologies, the platform provides a "What You See Is What You Get" (WYSIWYG) experience that matches the intuitiveness of modern design tools. Simultaneously, the system generates clean, production-ready code in the background to ensure technical excellence. TWB specifically addresses traditional layout constraints through a hybrid engine capable of handling both freeform absolute positioning and structured responsive flows, ensuring seamless compatibility across all devices.

1.2 The Problem Statement

Despite website builders being available, many users still have a hard time creating truly unique digital experiences. The main problems are:

1. **Layout issues:** Most builders only allow you to arrange things in a fixed grid. If you want to move something a little outside of that, it's either not possible or you have to mess around with complicated CSS code.
2. **Hard to learn:** The tools that offer a lot of flexibility often have interfaces that are super overwhelming. It takes months to figure them out, which scares off people who aren't tech-savvy.
3. **Code problems:** Builders that are easy to use often produce code that's messy and slows things down. It's also hard to take that code and use it on your website.
4. **Previews:** Most builders don't let you interact with your design in real time. You can't see how things will look when you hover over them or click on them, which means your design doesn't always match what users will actually see.

Many website builders exist. Users still struggle to create unique digital experiences.

The primary problems users face are:

- **Layout constraints**
- **Learning curves**
- **Code bloat**
- **Static previews**

1.3 Research Objectives

The objective behind developing the TWB — Template Website Builder is to overcome the drawbacks of traditional website building systems and to provide a more flexible, efficient, and high-performance platform that offers professional web design facilities to creators over the internet or offline.

The project that we developed also aims to overcome the shortcomings of current drag-and-drop builders and enhance current functions and develop new functions in an effort to make professional web development accessible and eliminate the need for users to have deep technical coding expertise or manage complex environments manually.

The developed system offers everything creators need, from precise layout control using a Hybrid Positioning Engine to real-time interactive previews and the automatic generation of clean, production-ready code to ensure the final product matches the design perfectly. The system also provides the user with everything they need to structure their site and make sure everything is going in straight lines regarding responsiveness and performance.

The system objectives could be summarized by the following:

1. Provide a flexible and secure design environment.
2. Bridge the gap between freeform design and responsive flows.
3. Reduce web development time and effort.
4. Better Performance through optimized code generation.
5. Real-time interaction and state previewing.
6. Empowering users with professional-grade styling tools.

1.4 Project Organization

This research is systematically partitioned into four chapters, providing a cohesive progression from theoretical conceptualization to technical implementation and terminal evaluation:

1. **Chapter One: *Introduction*** – Delineates the foundational research landscape by identifying critical bottlenecks in current web authoring paradigms, formulating a precise problem statement, and outlining the technical objectives required to synthesize design flexibility with architectural integrity.
2. **Chapter Two: *Theoretical Background and Technical Framework*** – Elucidates the technological ecosystem underpinning the project, specifically analyzing the MERN stack architecture, the implications of non-blocking I/O operations, and the utility of NoSQL document-oriented modeling for representing dynamic, deeply nested layout schemas.
3. **Chapter Three: *System Design and Implementation Methodology*** – Expounds upon the core engineering workflows of the platform, including the formalization of the "Hybrid Positioning Engine" algorithm, the implementation of deterministic state management protocols, and the deployment of RESTful interfaces for secure data persistence.
4. **Chapter Four: *Evaluation, Conclusion, and Future Directions*** – Provides a terminal assessment of the system's performance against the initial design requirements, summarizes the principal technical contributions, and identifies theoretical avenues for further optimization in the field of automated web development.

CHAPTER TWO

Background for Project

Chapter Two

Literature Review

2.1 Introduction

The development of modern web applications has shifted significantly from static HTML pages to dynamic, interactive, and component-based single-page applications (SPAs). Building a complex platform like a Template-Based Website Builder requires a robust architecture capable of handling intensive operations such as real-time Document Object Model (DOM) manipulation, state management, and seamless client-server communication. This chapter provides a general overview of the foundational concepts, programming languages, tools, and database systems utilized to construct this project. It highlights the rationale behind selecting specific technologies over others to achieve high performance and full creative freedom for the end-user.

Chapter Two

Literature Review

2.2 Languages

The project fundamentally relies on the core languages of the web to ensure cross-browser compatibility and high performance.

2.2.1 JavaScript

JavaScript serves as the core execution engine facilitating the "Logical Layer" of the entire system. By adopting an isomorphic (full-stack) JavaScript approach via the MERN stack, the project achieves seamless data serialization between the client and server [1]. In the context of the TWB project, JavaScript is utilized to manage the complex, recursive state of the website builder, handling intensive asynchronous event loops for drag-and-drop interactions and real-time Document Object Model (DOM) updates without requiring full-page reloads.

2.2.2 HTML (HyperText Markup Language)

Within this system, HTML5 is viewed not merely as static markup, but as the dynamic rendering target for the application's internal state. The builder utilizes HTML5's semantic structural elements to ensure that the dynamically generated code follows modern web standards for SEO and accessibility [2]. Furthermore, the project leverages the HTML5 DOM API to facilitate the "Live Preview" feature, where the virtual layout state is instantly converted into a standardized document structure.

2.2.3 CSS (Cascading Style Sheets)

CSS3 is employed as the "Layout Specification Language," defining the visual boundaries and spatial relationships between components. The project specifically focuses on the "Synthesis of Positioning Paradigms," combining advanced layouts like Flexbox and CSS Grid for structural containment with Absolute Positioning for freeform dragging. This hybrid approach allows the builder to calculate and apply styles dynamically, using CSS as the interpolation layer between the user's spatial input and the final production-ready output [3].

Chapter Two

Literature Review

2.3 Frameworks and Libraries

To accelerate development, ensure high performance, and maintain a scalable architectural foundation, several powerful JavaScript frameworks and libraries were incorporated into the system. These specific frameworks were chosen for their proven capabilities in managing highly intensive user interactions and data-handling tasks.

2.3.1 React.js (Declarative UI Library)

React.js acts as the primary "View Orchestrator" for the TWB platform, predicated on a declarative programming paradigm where the user interface is a direct mathematical function of the application state [4]. Within the Template-Based Website Builder, React is utilized to abstract the complexities of manual DOM manipulation into a structured, state-driven architecture.

The following technical paradigms within React are critical to the system's performance:

- **Component-Based Modularity:** The system decomposes the complex builder interface into isolated, autonomous modules (e.g., Canvas, Sidebar, ComponentProperties). Each component maintains encapsulated logic, facilitating a highly maintainable and scalable codebase while allowing for the granular rendering of specific layout elements [4].
- **Virtual DOM and Diffing Algorithm:** For high-frequency interactive tasks, such as dragging elements across the canvas, the traditional DOM's performance bottlenecks are bypassed using React's Virtual DOM. By employing a sophisticated "Reconciliation" algorithm (Diffing), the system calculates the minimal set of changes required to update the layout coordinates (X, Y), effectively preventing "Layout Thrashing" and ensuring a fluid 60-FPS user experience [4].
- **Unidirectional Data Flow and State Hooks:** To maintain a "Single Source of Truth" for complex website layouts, the project utilizes React Hooks (useState, useCallback, useMemo). This architecture ensures predictable state transitions where changes to a component's property (e.g., opacity or font size) propagate through the system in a unidirectional flow, maintaining data integrity across the editor, properties panel, and live preview [5].

Chapter Two

Literature Review

2.3.2 Node.js and Express.js

For the backend architecture, the system utilizes Node.js paired with the Express.js framework to create a highly responsive and scalable server environment.

- **Node.js (Event-Driven Architecture):** Node.js is a cross-platform runtime environment that executes JavaScript code server-side. Unlike traditional multi-threaded server models (like Apache), Node.js operates on a single-threaded, non-blocking, event-driven architecture [5]. This design is exceptionally efficient for handling numerous concurrent Input/Output (I/O) operations. In the website builder, when users save projects, the application transmits large JSON trees (representing the layout schema) to the server. Node.js processes these intense data payloads asynchronously, ensuring the server remains responsive without buffering [5].
- **Express.js (Routing and Middleware):** Paired tightly with Node.js, Express.js acts as a minimal yet powerful web framework. It provides a robust set of features for web applications, primarily acting as the routing layer. In this project, Express is utilized to construct highly secure RESTful APIs [1]. It simplifies the implementation of "Middleware" functions—such as parsing incoming JSON requests, establishing secure cross-origin resource sharing (CORS) policies, and authenticating user requests—facilitating secure and rapid communication between the React frontend editor and the database.

Chapter Two

Literature Review

2.4 Development Tools

To facilitate the creation, debugging, and testing of the application, as well as to implement complex interactions safely, several development tools, environments, and specialized libraries were utilized throughout the project lifecycle.

2.4.1 Visual Studio Code

Visual Studio Code (VS Code) is a powerful, lightweight source code editor. It was adopted as the primary Integrated Development Environment (IDE) for its extensive ecosystem of extensions, seamless Git integration, and robust support for the full-stack JavaScript environment [8].

2.4.2 Google Antigravity

Google Antigravity is an advanced agentic AI coding assistant that integrates directly into the development workflow. It was utilized for intelligent code generation, rapid refactoring, debugging complex logic, and assisting in system design, significantly enhancing overall developer productivity [9].

2.4.3 Neovim

Neovim is a highly extensible, terminal-based text editor. It was employed for rapid script editing, server configurations, and efficient shell-based workflows, offering a fast, keyboard-centric environment for quick code modifications [10].

2.4.4 Specifications of the PC used for development:

1. **Processor:** AMD Ryzen™ 5 PRO 5650G @3.9 GHz, Up to @4.4 GHz.
2. **Motherboard:** MSI B450 Gaming Pro Carbon Max WiFi.
3. **Graphics Card:** AMD Radeon™ RX 580 8G Graphics Card.
4. **RAM:** 32GB (3200 MHz).
5. **Disk Storage:** M.2 (256 GB) and SSD (1T).
6. **Operating System:** Arch Linux.

Chapter Two

Literature Review

2.4.5 Web Browsers (Google Chrome & Firefox)

Testing the application and its generated layouts across multiple modern rendering engines is critical, as debugging is deeply context-dependent [11].

Google Chrome: Served as the primary browser for development and live debugging, leveraging the powerful Chrome DevTools to inspect DOM elements, monitor network activity, and analyze React component states.

Mozilla Firefox: Utilized to ensure cross-browser compatibility. Its specialized developer tools provide unique advantages, particularly for deeply inspecting and tweaking complex CSS Grid and Flexbox layouts [11].

2.4.6 React Drag and Drop (React DnD)

A crucial aspect of any modern website builder is the ability to select, move, and place elements on a canvas intuitively. React DnD is a powerful set of React utilities that provides high-level abstractions for handling complex drag-and-drop interfaces. Unlike standard HTML5 drag-and-drop APIs, React DnD allows for isolated, state-driven interactions. It makes it possible to seamlessly transfer data (such as dragged component types and unique IDs) across different sections of the workspace. This library forms the backbone of the Hybrid Positioning Engine, ensuring smooth interactions, accurate dropping calculations, and visual updates for the component layout in real time [6].

2.4.7 Version Control (Git & GitHub)

Robust source code management and collaborative capabilities are fundamental to modern software engineering.

- **Git:** Employed as the primary distributed version control system (VCS). It enables tracking granular changes across the codebase, managing different software versions, and safely reverting to previous states if issues arise.
- **GitHub:** Utilized as the cloud-based hosting platform for the Git repository. It facilitates seamless collaboration, code review via pull requests, issue tracking, and comprehensive project management [12].

Chapter Two

Literature Review

2.5 Database Design

Given the nature of website builders, user projects consist of highly nested, varied data structures (like HTML trees and CSS styles) rather than rigid, uniform data blocks. Therefore, a flexible data storage solution is required.

2.5.1 MongoDB

MongoDB is a leading NoSQL, document-oriented database system that stores data in flexible, JSON-like documents (BSON). This structure perfectly aligns with the website builder's core architecture [7]. In this project, every page or template created by a user is essentially represented as a hierarchical JSON tree containing UI elements, node properties, and styles. Using MongoDB allows the backend to store, query, and update these complex page structures natively and efficiently without the need for strict, predefined relational SQL tables. This ensures that massive templates and deeply customized user projects load quickly and can be saved reliably [7].

Core Collections (Database Tables) The system's database schema is designed around three primary collections to manage users, their ongoing projects, and reusable layout templates:

Table 1: User Collection

(Manages authentication and user profiles).

Field	Data Type (Length)	Content
name	String (min 2)	The user's full name.
email	String	The user's email address (must be unique).
password	String (min 6)	The user's password, securely hashed via bcrypt.
avatar	String	A URL or path to the user's profile picture.

Chapter Two

Literature Review

Implementation (Mongoose Schema):

```
const mongoose = require('mongoose');
```

```
const bcrypt = require('bcryptjs');
const UserSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Name is required'],
    trim: true,
    minlength: [2, 'Name must be at least 2 characters']
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    lowercase: true,
    trim: true,
    match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address']
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [6, 'Password must be at least 6 characters']
  },
  avatar: {
    type: String,
    default: ""
  }
}, {
  timestamps: true
});
```

Chapter Two

Literature Review

```
// Hash password before saving
UserSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Compare password method
UserSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', UserSchema);
```

Table 2: Project Collection

(Acts as the core storage for individual websites created by users).

Field	Data Type (Length)	Content
title	String	The name of the project.
userId	ObjectId	A reference linking the project to its creator (User).
data	Object (JSON)	A flexible JSON object field that stores the entire deeply nested layout structure (UI elements, properties, absolute coordinates) of the website.
isPublished	Boolean	A flag indicating if the website is currently live.

Chapter Two

Literature Review

Implementation (Mongoose Schema):

```
const mongoose = require('mongoose');
```

```
const ProjectSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
  },
  data: {
    type: Object, // To store page structure (JSON) and drag-and-drop elements
    default: {}
  },
  isPublished: {
    type: Boolean,
    default: false
  }
}, {
  timestamps: true
});
module.exports = mongoose.model('Project', ProjectSchema);
```


Chapter Two

Literature Review

Table 3: Template Collection

(Stores pre-designed, ready-to-use component layouts that users can drop into their projects.)

Field	Data Type (Length)	Content
name	String	The unique identifier or name of the template.
components	Array	An array storing the structural JSON of the template elements.

Implementation (Mongoose Schema):

```
const mongoose = require('mongoose');
```

```
const TemplateSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    unique: true
  },
  components: {
    type: Array, // To store the array of component objects
    default: []
  }
}, {
  timestamps: true
});
module.exports = mongoose.model('Template', TemplateSchema);
```

CHAPTER THREE

Design and Implementation

Chapter Three

Methodology

3.1 Design and Implementation

The methodology adopted for this project revolves around modern web development practices, specifically utilizing a component-based architecture via React.js and a robust backend using the MERN stack (MongoDB, Express.js, React, Node.js). The primary design philosophy of the website builder is to offer a "Hybrid Capabilities" model. Traditional builders generally fall into two categories: strict grid/flow-based (like Webflow) and absolutely free-drag (like Wix). In this system, both approaches are synthesized. Why was this done? To provide advanced users with the structured capabilities of CSS Flexbox and Grid for responsive sections while still retaining the freedom to drag and drop overlapping elements absolutely using X and Y coordinates.

Chapter Three

Methodology

3.2 General Structure

The system's architecture follows a classic Client-Server (Frontend-Backend) model:

- **Frontend (Client):** Built exclusively as a Single Page Application (SPA). The state of the entire project (including multiple pages, nested HTML components, and deeply modified CSS styles) is held in memory using React Hooks (`useState`, `useCallback`, `useEffect`). Global context providers (`AuthContext`, `DndProvider`) wrap the application to manage session states and drag-and-drop contexts efficiently.

```
// App.js - Entry Point & Providers
```

```
return (  
  <AuthProvider>  
    <DndProvider backend={HTML5Backend}>  
      <Editor />  
    </DndProvider>  
  </AuthProvider>  
)
```

- **Backend (Server):** A Node.js environment running Express handles RESTful API requests. The choice to use a NoSQL database (MongoDB) was critical here. Since a user's website layout is essentially a massive, varying JSON tree containing elements and their styles, saving this data in rigid SQL tables would require complex, slow joins. MongoDB naturally accepts these JSON structures directly into the Project collection (`data` field), making load and save operations remarkably fast.

// Project.js Model - NoSQL JSON Storage

```
const ProjectSchema = new mongoose.Schema({  
  title: String,  
  data: {  
    type: Object, // Stores the massive JSON layout tree  
    default: {}  
  }  
});
```

3.2.1 Stateless Authentication (JWT)

To secure the Client-Server communication without degrading the performance of the SPA, Stateless Authentication using JSON Web Tokens (JWT) was implemented. Unlike traditional Session-ID approaches that force the server to repeatedly query the database (increasing server overhead), JWTs carry the user's verified claims (like `userId`) inside a cryptographically signed payload [14]. This ensures data integrity and scalability, allowing the frontend to quickly verify user sessions mathematically without waiting on database latency [15].

```
// server/utils/authUtils.js - Stateless Token Generation
```

```
const generateToken = (userId) => {  
  
  // Encapsulate the userId into a signed payload  
  
  return jwt.sign({ id: userId }, process.env.JWT_SECRET, {  
  
    expiresIn: '30d' // Token remains valid for 30 days  
  
  });  
};
```

Communication between the two layers occurs asynchronously via Axios, with an automated debounced saving mechanism introduced on the frontend to prevent data loss without overwhelming the server with continuous requests on every pixel movement.

Chapter Three

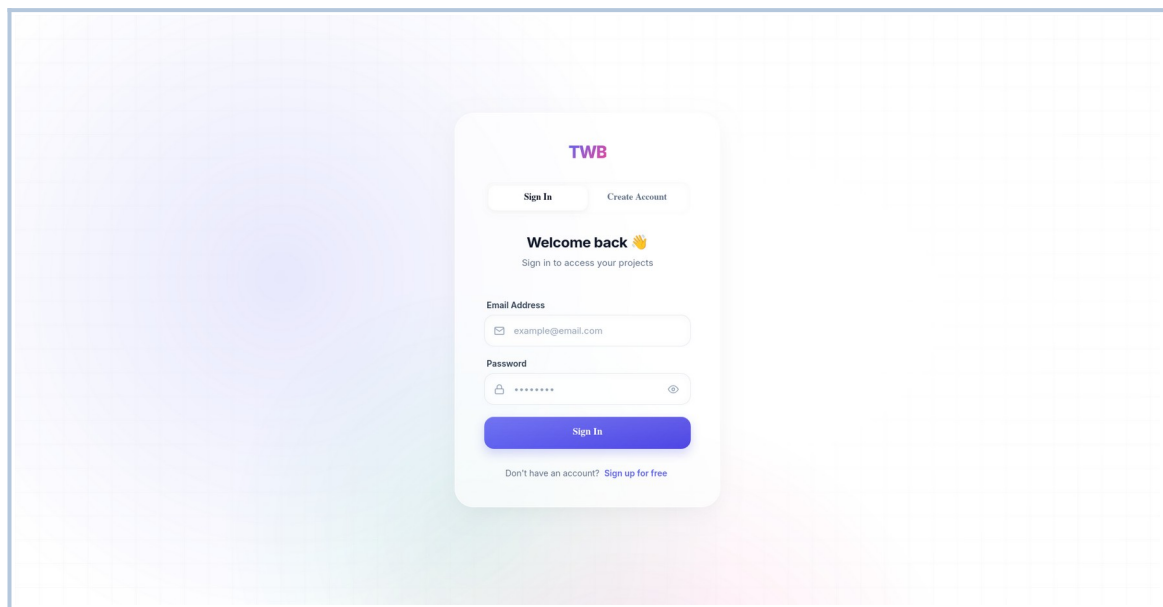
Methodology

3.3 User Interfaces and Pages

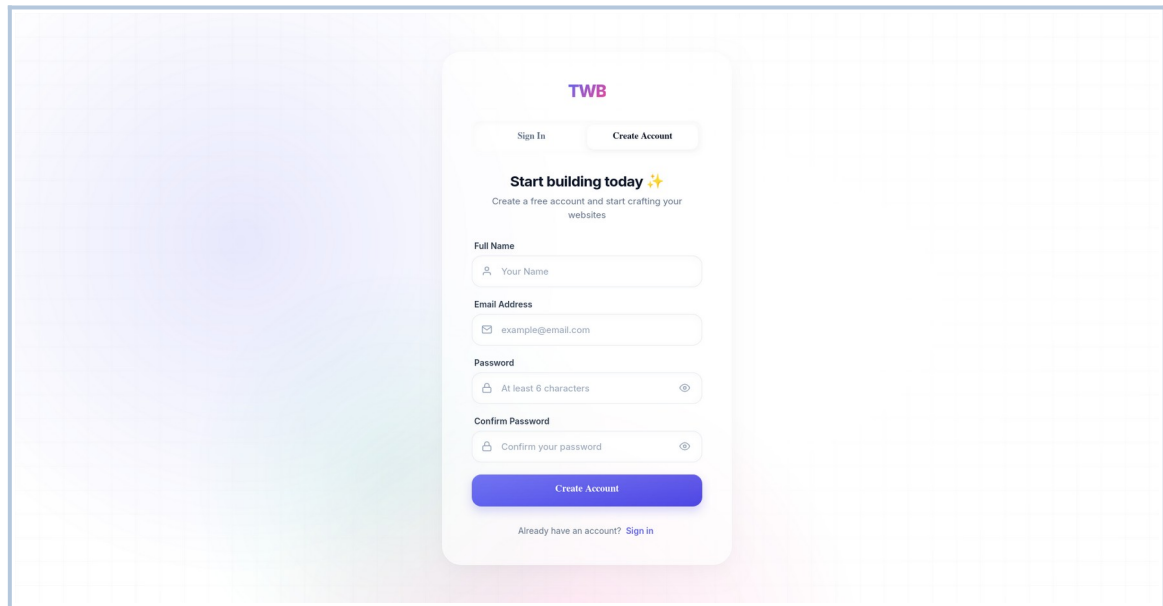
The system is heavily modularized, rendering different "pages" or "views" based on user interaction securely bounded by authentication.

3.3.1 User Page (Authentication and Dashboard)

Access to the builder tool requires user identification. The `AuthPage` serves as the gateway, handling user registration and login securely.



Screenshot 1: User Authentication.



Screenshot 2: User Registration Interface.

Design Justification: We implemented an authentication barrier (via `PrivateRoute` bindings) because website data is persistent and personal. By tying a `userId` to a `Project` document in MongoDB, the system ensures data privacy and allows users to seamlessly resume their work from any device. A dropdown User Menu was also designed to give quick access to account details and sign-out functionality.

Implementation Logic (Frontend Validation): Before the data is transmitted to the server, the frontend performs a deterministic check to ensure data integrity:

// client/src/pages/AuthPage/AuthPage.js - Validation Logic

```
if (formData.password !== formData.confirmPassword) {  
  setLocalError('Passwords do not match');  
  setIsLoading(false);  
  return;  
}
```


Implementation Logic (Backend Authentication): The following server-side controllers manage the security lifecycle of a user session, from initial registration to secure login verification.

// server/controllers/authController.js - Registration & Login Logic

```
const register = async (req, res) => {
  const { name, email, password } = req.body;
  const existingUser = await User.findOne({ email });
  if (existingUser) return res.status(400).json({ message: 'Email already registered' });

  const user = await User.create({ name, email, password });
  const token = generateToken(user._id);
  res.status(201).json({ token, user: { id: user._id, name, email } });
};

const login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (user && (await user.matchPassword(password))) {
    const token = generateToken(user._id);
    res.status(200).json({ token, user: { id: user._id, name: user.name, email } });
  } else {
    res.status(401).json({ message: 'Invalid credentials' });
  }
};
```

To protect private routes, a centralized middleware validates the JSON Web Token (JWT) in the request header:

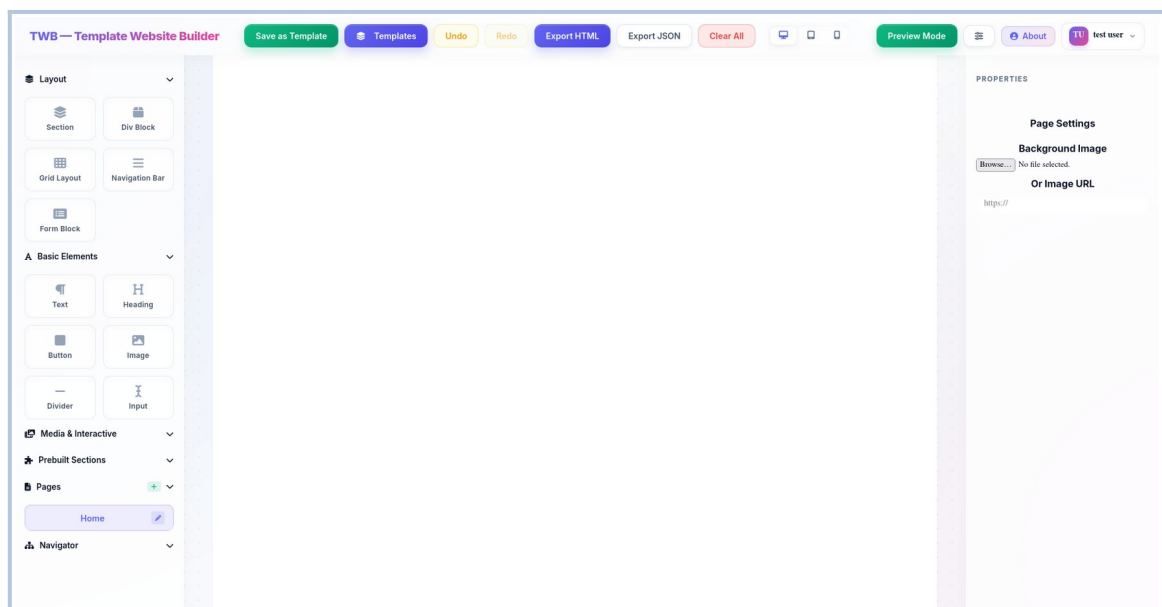
// server/middleware/authMiddleware.js

```
const protect = async (req, res, next) => {  
  
  let token = req.headers.authorization?.split(' ')[1];  
  
  if (!token) return res.status(401).json({ message: 'No Token' });  
  
  try {  
  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
  
    req.user = await User.findById(decoded.id).select('-password');  
  
    next();  
  
  } catch (err) {  
  
    res.status(401).json({ message: 'Invalid Session' });  
  
  }  
  
};
```

3.3.2 Editor Page

The core workspace (`Editor.js`) is the most computationally heavy page in the system. It is divided into three functional zones:

1. **The Sidebar/Toolbar:** Contains draggable elements categorized logically (Basic, Media, Sections) and a Page Manager. The Page Manager allows users to add, rename, or delete multiple sub-pages (e.g., Home, Contact) within a single project, dynamically updating the React state without refreshing the browser.
2. **The interactive Canvas:** Acts as the drop zone (`useDrop`). It calculates exact mouse coordinates, applying an optional "Grid Snapping" math algorithm to align elements perfectly.
3. **The Properties Panel:** A dynamic panel that reflects the CSS properties of the currently selected element.



Screenshot 3: Full Editor Workspace Structure (Sidebar, Canvas, Properties).

Design Justification: To improve User Experience (UX), a floating toolbar was implemented over selected elements, allowing quick actions like duplication or deletion without having to travel across the screen to the sidebar. Furthermore, an Undo/Redo stack (history and future states) was explicitly built to track changes to the canvas state, forgiving user mistakes instantly.

Implementation Logic (Editor Engine): The Editor's performance and alignment accuracy are maintained via several low-level algorithmic implementations:

1. Grid Snapping and Drop Calculations: To ensure geometric precision, the useDrop handler intercepts raw pointer coordinates and rounds them to the nearest grid step.

// client/src/components/Editor/Editor.js - Grid Snapping Logic

```
const gridSize = 10;

let x = (clientOffset.x - canvasRect.left - borderLeft) + scrollLeft;

let y = (clientOffset.y - canvasRect.top - borderTop) + scrollTop;

x = Math.round(x / gridSize) * gridSize;

y = Math.round(y / gridSize) * gridSize;
```

2. Interactive State Management (Hover/Active): Components dynamically resolve their styles based on user interactions, allowing for a "Live" design experience.

// client/src/components/Editor/CanvasItem.js - Style Resolution Engine

```
const getActiveStyle = () => {

    let finalStyle = { ...comp.style };

    if (isHovered && comp.states?.hover) finalStyle =
{ ...finalStyle, ...comp.states.hover };

    if (isActive && comp.states?.active) finalStyle = { ...finalStyle, ...comp.states.active };

    return finalStyle;

};
```

3. Multi-Device Responsive Logic: Styles are isolated per breakpoint, ensuring that mobile-specific changes do not affect the desktop layout.

// client/src/components/Editor/Editor.js - Responsive Breakpoint Logic

```
const currentStyle = viewMode === 'desktop'  
  ? comp.style  
  : { ...comp.style, ...(comp.responsiveStyles?.[viewMode] || {}) };
```

4. Debounced Persistence (Auto-Save): To prevent server overload while ensuring data safety, an automated saving mechanism triggers 1000ms after the last canvas modification.

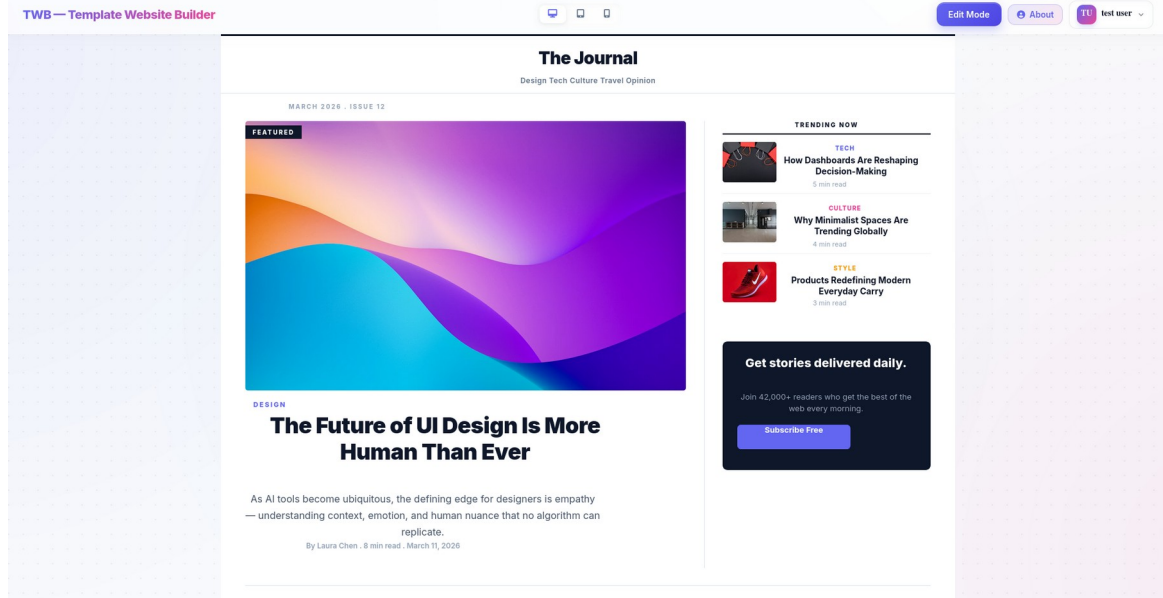
// client/src/components/Editor/Editor.js - Debounced Persistence Logic

```
useEffect(() => {  
  const debounceTimer = setTimeout(() => {  
    saveProjectToServer(pages);  
  }, 1000);  
  return () => clearTimeout(debounceTimer);  
}, [pages]);
```

3.3.3 Preview Page (Mode)

Rather than redirecting the user to a completely separate URL to view their site, a previewMode state toggle is built directly into the Editor.

Screenshot 4: Preview Mode simulating final website rendering.



Design Justification: Activating Preview Mode completely hides the sidebar, properties panel, bounding boxes, and grid lines. This decision was made to drastically reduce the feedback loop time. The user can see the exact, final rendered HTML/CSS output instantly, assessing responsive design modes (Desktop, Tablet, Mobile) without the heavy loading times of a traditional external preview link.

Implementation Logic (Export Engine): The platform utilizes a serialization logic that compiles the complex React state into semantic, standalone HTML/CSS.

1. Dynamic HTML Generation: This recursive function traverses the component tree, converting style objects into inline CSS and assigning unique IDs for targeted styling.

// client/src/utils/exportUtils.js - Recursive HTML Serializer

```
const generatePreviewHTML = (components) => {  
  return components.map(comp => {  
    const styleStr = Object.entries(comp.style).map(([k, v]) =>  
      `${k.replace(/[A-Z]/g, m => '-' + m.toLowerCase())}:${v}`).join(';');  
    return `  }).join("\n");  
};
```

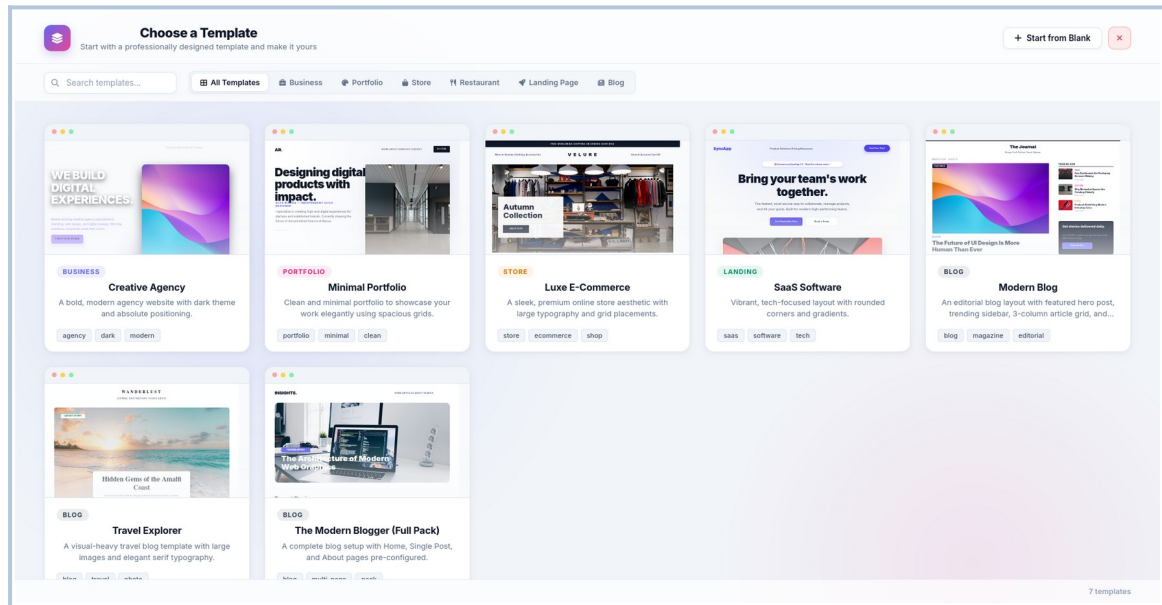
2. Production Export: The system bundles the generated HTML with a consolidated CSS block and triggers a browser download using a Blob object, making the design portable.

// client/src/components/Editor/Editor.js - Production Export Engine

```
const handleExportHTML = () => {  
  const fullHTML = `<!DOCTYPE html>...<style>${styles}</style><body>${body}</body></html>`;  
  const blob = new Blob([fullHTML], { type: 'text/html' });  
  const url = URL.createObjectURL(blob);  
  const link = document.createElement('a');  
  link.href = url;  
  link.download = 'exported_site.html';  
  link.click();  
};
```

3.3.4 Templates Page (Template Gallery)

The Template Gallery is integrated as a modal/overlay page that presents users with pre-designed layouts.



Screenshot 5: The internal Template Gallery selection screen.

Design Justification: Analyzing user retention in drag-and-drop builders revealed that users often face "blank canvas paralysis." By providing a database-driven **TemplateCollection**, users can inject professionally designed, pre-built component arrays directly into their workspace. This jumpstarts the design process and acts as a learning tool for how complex grids and flexboxes can be constructed.

Implementation Logic (Templates & Portability): Saving and loading templates involves a transformation layer that ensures newly injected components receive unique IDs and consistent positioning.

1. **Template Persistence:** Layouts are stored as raw JSON objects within a shared MongoDB collection.

// client/src/components/Editor/Editor.js - Template Database Persistence

```
const saveTemplate = async (templateData) => {  
  await axios.post('/api/templates', {  
    name: templateData.name,  
    components: templateData.activeComponents  
  });  
};
```

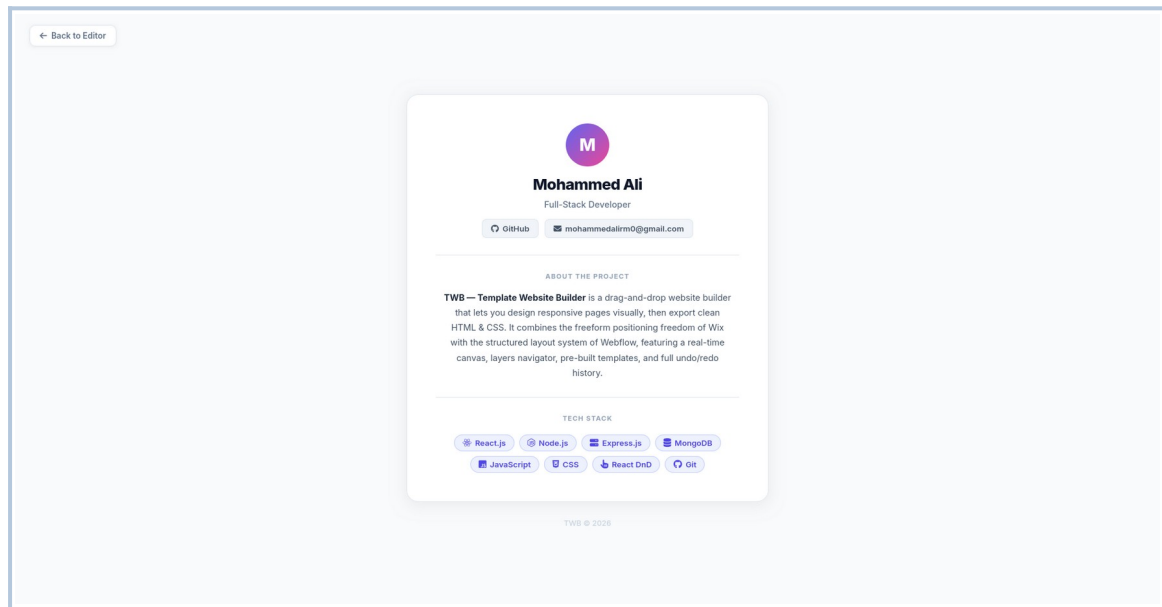
2. **Component Normalization:** When a template is loaded, IDs are regenerated based on timestamps to prevent collisions within the current project state.

// client/src/utils/templateUtils.js - Component Normalization

```
const normalizeComponents = (comps) => {  
  return comps.map((comp, idx) => ({  
    ...comp,  
    id: Date.now() + idx, // Ensure global uniqueness  
    style: { ...comp.style, position: 'absolute' }  
  }));  
};
```

3.3.5 About Page

The AboutPage serves as informational and instructional documentation accessible directly from the application workspace.



Screenshot 6: About Page.

Design Justification: Instead of hosting documentation externally, placing the About page within the SPA routing ensures that users can quickly learn about system, or project details without disrupting their session flow or losing their unsaved progress.

3.4 State Management and Algorithmic Logic

3.4.1 React State Management

Handling the state of an entire website layout within a browser's memory is computationally expensive. The state of the project (multiple pages, nested HTML components, and deeply modified CSS properties) is held using React Hooks (`useState`, `useCallback`) and Global Contexts. Instead of using heavy global stores like Redux, which increase bundle size and memory footprint, the local component tree manages the updates.

// client/src/components/Editor/Editor.js - Complex State Structure & Update Pattern

```
const [pages, setPages] = useState([  
  id: 'home',  
  name: 'Home',  
  components: [], // Array of CanvasItem objects  
  style: { backgroundColor: '#ffffff' }  
]);  
  
const updateComponentStyles = useCallback((id, newStyles) => {  
  setPages(prevPages => prevPages.map(page => ({  
    ...page,  
    components: page.components.map(comp =>  
      comp.id === id ? { ...comp, style: { ...comp.style, ...newStyles } } : comp  
    )  
  })));  
}, [activePageId]);
```

The use of the Virtual DOM ensures that when a user drags an element or types in a text box, React only re-renders the specific component (`CanvasItem`) that changed, preserving the 60-FPS smoothness required for a dragging interface [16].

3.4.2 Grid Snapping and Drop Calculations

The drag-and-drop mechanism is not purely visual; it involves an underlying spatial algorithm. When a user drags an element, the `useDrop` hook calculates the exact pointer coordinates (X, Y) relative to the `Canvas` bounding box. To ensure structural alignment, a Grid Snapping algorithm intercepts these coordinates before rendering:

```
x = Math.round(x / gridSize) * gridSize
```

// client/src/components/Editor/Editor.js - Spatial Coordinate Calculation Logic

```
const [, drop] = useDrop({
  accept: 'ITEM',
  hover(item, monitor) {
    const clientOffset = monitor.getClientOffset();
    const canvasRect = canvasRef.current.getBoundingClientRect();

    // Calculate raw position
    let rawX = clientOffset.x - canvasRect.left;
    let rawY = clientOffset.y - canvasRect.top;

    // Apply Mathematical Snapping Logic
    const snappedX = Math.round(rawX / gridSize) * gridSize;
    const snappedY = Math.round(rawY / gridSize) * gridSize;

    moveComponent(item.id, snappedX, snappedY);
  }
});
```

This mathematical coercion forces randomly dragged elements into a strict, invisible geometric grid (e.g., 10px by 10px), eliminating the misalignment issues commonly found in absolute free-drag editors.

CHAPTER FOUR

Conclusion and Future Work

CHAPTER FOUR

Conclusion and Future Work

4.1 Conclusions

The development of the Template Website Builder (TWB) has successfully demonstrated that the gap between high-level design flexibility and structured web development can be bridged through a robust "Hybrid Capabilities" model. By synthesizing the absolute positioning freedom typical of graphic design tools with the structured flow of modern CSS (Flexbox and Grid), the system provides an intuitive yet powerful workspace for both novice users and professional designers.

Key conclusions drawn from this project include:

- **Efficiency of the Hybrid Engine:** The implementation of a spatial algorithm for grid snapping and coordinate calculation allows for precise layout control while maintaining the responsiveness required for modern web environments. The mathematical coercion logic handles the complexity of absolute-to-relative coordinate conversion seamlessly.
- **Performance-Driven SPA:** Utilizing React.js and its Virtual DOM architecture ensures a high-performance, 60-FPS editing environment. This low-latency feedback loop is essential for a professional-grade drag-and-drop experience.
- **Scalable Data Architecture:** The choice of a NoSQL database (MongoDB) was instrumental in managing the non-linear, hierarchical JSON data structures that represent modern website layouts. This ensures rapid saving and loading of complex projects.
- **Stateless Security:** Implementing JWT-based authentication provided a secure method for managing user sessions without adding overhead to the backend, ensuring a smooth transition between the dashboard and the heavy editor interface.
- **User-Centric Design:** The project successfully integrated a Template Gallery and Preview Mode, significantly reducing the learning curve and helping users overcome the friction of starting from scratch.

In summary, the project has achieved its primary research objectives, delivering a functional, flexible, and scalable platform that simplifies the web design process while maintaining creative control.

CHAPTER FOUR

Conclusion and Future Work

4.2 Future Work

Although the TWB project has established a strong foundation, there are several avenues for further development and research:

- **Full-Site Deployment & Asset Optimization:** Future work should include a "One-Click Publish" feature that optimizes images, minifies CSS/JS, and deploys the project directly to edge hosting services (like Vercel or Netlify).
- **AI-Driven Design Assistance:** Integrating Generative AI models could help users generate content, recommend harmonious color palettes, and even suggest layout modifications based on best practices for specific industries.
- **Real-Time Collaboration:** Expanding the architecture to support WebSockets (via Socket.io) would enable multiple designers to work simultaneously on a single project, fostering collaborative creativity.
- **Component Marketplace & Plugins:** Creating a framework for third-party developers to contribute "Plugins" or custom components would exponentially grow the builder's feature set (e.g., e-commerce integrations, booking systems).
- **Interactive Prototyping:** Adding advanced state management for animations and page transitions would allow the TWB to function as a professional prototyping tool, similar to Framer or Figma.
- **Mobile Design App:** Developing a lightweight mobile version of the dashboard for tracking project analytics and making minor content updates on the go.

The TWB project serves as a starting point for the next generation of intelligent, accessible web building tools, with the potential to democratize high-end web design for everyone.

REFERENCES

- [1] E. Brown, *Web Development with Node and Express: Leveraging the JavaScript Stack*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [2] J. Duckett, *HTML and CSS: Design and Build Websites*, 1st ed. Indianapolis, IN, USA: Wiley, 2011.
- [3] E. A. Meyer and E. Weyl, *CSS: The Definitive Guide*, 4th ed. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [4] V. M. Ganeshan, "Web Development Using ReactJS," in *Proc. 5th Int. Conf. Adv. Comput., Commun. Control Netw. (ICAC3N)*, 2023, pp. 1–5.
- [5] S. Tilkov and S. Vinoski, "Node.js: Using JavaScript to Build High-Performance Network Programs," *IEEE Internet Comput.*, vol. 14, no. 6, pp. 80–83, Nov.–Dec. 2010.
- [6] A. Banks and E. Porcello, *Learning React: Modern Patterns for Developing React Apps*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [7] Y. Li and S. Manoharan, "A performance comparison of SQL and NoSQL databases," in *Proc. IEEE Pacific Rim Conf. Commun., Comput. Signal Process. (PACRIM)*, 2013, pp. 15–19.
- [8] J. Tan, Y. Chen, and S. Jiao, "Visual Studio Code in Introductory Computer Science Course: An Experience Report," in *Proc. ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE)*, 2023.
- [9] IEEE Computer Society, "The Impact of AI on Productivity and Code in Software Engineering," *IEEE Softw.*, vol. 41, no. 1, 2024.
- [10] B. Perez et al., "Learning the State Machine Behind a Modal Text Editor: The (Neo)Vim Case Study," *IEEE Trans. Softw. Eng.*, 2024.
- [11] M. Wang et al., "How Developers Choose Debugging Strategies for Challenging Web Application Defects," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng. (ICSE)*, 2024.
- [12] N. Dinh et al., "GitHub Analytics to Support Collaboration in Software Engineering Teams," *IEEE Trans. Comput. Social Syst.*, 2024.
- [13] P. Spinellis, "Git," *IEEE Softw.*, vol. 29, no. 3, pp. 100–101, May–Jun. 2012.
- [14] J. Nurmi et al., "A JSON Token-Based Authentication and Access Management Schema for Cloud SaaS Applications," in *Proc. IEEE 5th Int. Conf. Future Internet of Things and Cloud (FiCloud)*, 2017, pp. 3–10.
- [15] R. M. Ahmed and M. Mahmood, "An authentication based scheme for applications using JSON web token," in *Proc. 2nd Int. Conf. Comput. Inf. Sci. (ICCIS)*, 2019, pp. 1–6.
- [16] V. M. Ganeshan, "Web Development Using ReactJS: Virtual DOM Performance in SPAs," in *Proc. 5th Int. Conf. Adv. Comput., Commun. Control Netw. (ICAC3N)*, 2023.

الملخص

أصبح التواجد القوي والفعال على شبكة الإنترنت ضرورة أساسية لا غنى عنها للشركات والمبدعين والأفراد في العصر الرقمي الحديث. ومع ذلك، لا تزال عملية تطوير المواقع الإلكترونية تمثل تحدياً، حيث تنقسم عادةً بين بيئات البرمجة الاحترافية التي توفر مرونة مطلقة بمتطلبات تقنية معقدة، وبين أدوات بناء المواقع التقليدية التي تضحي بالحرية الإبداعية وجودة الأداء مقابل سهولة الاستخدام. يقدم هذا البحث مشروع أداة إنشاء مواقع الويب الجاهزة "TWB — Template Website Builder"، وهو منصة رائدة صُممت خصيصاً لسد هذه الفجوة من خلال توفير بيئة عمل متطورة تعتمد على مبدأ "ما تراه هو ما تحصل عليه" (WYSIWYG)، والتي تضع استقلالية المصمم والتميز التقني في صدارة أولوياتها. يتمثل الابتكار الأساسي في مشروع TWB في "محرك التموضع الهجين" (Hybrid Positioning Engine). فخلافًا للمنصات التقليدية التي تحصر المستخدمين في أطر شبكية صلبة، يتيح TWB الانتقال السلس بين التصميم الحر المعتمد على الإحداثيات المطلقة—مما يسمح بدقة متناهية في توزيع العناصر—وبين التدفقات الهيكلية المتجاوبة التي تضمن توافق الموقع مع مختلف أحجام الشاشات. يوفر هذا النهج تجربة تصميم بديهية تحاكي أدوات التصميم الجرافيكي الاحترافية، مع الحفاظ على البنية البرمجية المتينة للموقع. علاوة على ذلك، تعالج المنصة مشكلة "ترهل الكود البرمجي" الشائعة من خلال توليد كود نظيف، محسن، وجاهز للإنتاج الفوري. بالاعتماد على حزمة تقنية قوية تضم React.js لتطوير الواجهات الديناميكية، و Node.js و MongoDB لإدارة البيانات والبنية التحتية، يطبق المشروع ميزات تنافسية مثل مساحة العمل اللانهائية والمعاينة اللحظية للحالات التفاعلية. يخلص البحث إلى أن دمج هذه القدرات الاحترافية في بيئة تطوير "بدون كود" يساهم بفاعلية في تمكين المستخدمين من بناء تجارب رقمية متطورة، مما يوفر حلاً مثالياً يجمع بين سرعة الإنجاز والجودة العالية دون الحاجة إلى منحني تعلم برمجي معقد.

الكلمات المفتاحية: تطوير المواقع، بدون كود، السحب والإفلات، محرك التموضع الهجين، تصميم متجاوب، بناء المواقع، React.js، تجربة المستخدم.



وزارة التعليم العالي
والبحوث العلمي
جامعة القادسية
كلية علوم الحاسوب وتكنولوجيا المعلومات
قسم نظم المعلومات الحاسوبية



منشئ مواقع ويب قائم على القوالب

مشروع مقدم الى قسم نظم المعلومات الحاسوبية كجزء من متطلبات نيل درجة
البكالوريوس في نظم المعلومات الحاسوبية

مقدم من:

محمد علي فليح

بأشراف الأستاذ:

د. الاء عبد مسلم عبد علي

آذار 2026